

PRODUCT REQUIREMENTS DOCUMENT

Garment Production Tracker

Version 2.0 — MVP

Product	Garment Production Tracker
Version	2.0 (MVP)
Author	Solo Developer
Stack	Next.js · Supabase · Tailwind · shadcn/ui

1. Objective

To build a web-based, mobile-responsive application to track a garment manufacturing order from initial Purchase Order (PO) to final delivery and payment. This system replaces manual spreadsheets and paper job cards.

The application is being built as a solo project. All frontend work uses Next.js (App Router) with Tailwind CSS and shadcn/ui for the component layer. The backend is entirely managed through Supabase, which provides the PostgreSQL database, authentication, file storage, realtime subscriptions, and Edge Functions.

2. Target Audience & Roles (RBAC)

Access is restricted to authorised factory staff via Role-Based Access Control (RBAC). In Supabase, each user's role is stored in their JWT metadata and enforced at the database level using Row Level Security (RLS) policies — not in application code.

Role	Primary Interface	Responsibilities
Super Admin	Web	System setup, user management, role assignment
Director	Web (read-only)	High-level analytics dashboard
Production Planner	Web	Enter orders, monitor schedules, manage BOM
Store / Cutting / Supervisors	Mobile (PWA)	Update inventory, track floor progress
Accounts Manager	Web	Generate invoices, track payments, close POs

3. Tech Stack

Layer	Technology
Framework	Next.js 14 (App Router, React Server Components)
UI / Styling	Tailwind CSS + shadcn/ui component library
Database	Supabase — PostgreSQL with Row Level Security
Authentication	Supabase Auth (JWT + role stored in user metadata)
File Storage	Supabase Storage (POD images, generated PDFs)
Realtime	Supabase Realtime subscriptions (live floor updates)
Server Logic	Supabase Edge Functions (Deno) — BOM calc, PDF gen

Hosting	Vercel (recommended) — native Next.js support
Offline Support	localStorage queue + navigator.onLine flush strategy

4. Key Features — Must-Haves

4.1 Authentication & Security

- Login portal using Supabase Auth (email/password for MVP)
- User role stored in auth.users metadata (super_admin, director, planner, floor, accounts)
- RLS policies on every Supabase table — access enforced at the DB layer, not the app layer
- Next.js middleware reads the Supabase session JWT and redirects unauthorized routes

4.2 Order Management (Web)

- Form to input PO details: Customer name, PO date, SKU list, sizes, quantities, delivery date
- Automatic BOM calculation: triggers a Supabase Edge Function (order_qty * material_per_item)
- BOM vs. inventory check: system flags a Purchase Request if stock is insufficient
- Orders list view with status column (Draft, In Production, QC, Dispatch, Closed)

4.3 Inventory Management

- Table view of current fabric and trims stock (material name, unit, quantity on hand)
- Stock deduction: triggered when Store Manager taps 'Release Material' on the floor view
- Low-stock visual indicator (highlight row in amber when quantity falls below threshold)

4.4 Floor Tracking (Mobile-first PWA)

- Queue of active job cards — one card per PO, showing current stage
- One-tap stage progression buttons (shadow Button components, large touch targets):
 - Release Material → Cutting Complete → Stitching Complete
 - Kaj & Buttoning Complete → Ironing Complete → Ready for Packing
- QC form: Pass / Fail radio buttons; if Fail, a required text field for rework notes
- Failed orders are automatically sent back to the Stitching stage in the queue

4.5 Logistics & Fulfillment

- Proof of Delivery (POD): mobile image upload to Supabase Storage
- Delivery Challan: generated as PDF via Edge Function (react-pdf or pdfmake), stored and linked
- Delivery tracking: free-text field to enter courier name and tracking number

- Payment ledger: Accounts Manager enters amounts received; system shows balance outstanding
- PO closure: when full payment is logged, Accounts Manager marks PO as Closed; it is archived

5. Non-Functional Requirements

Requirement	Implementation
Cross-platform	Responsive design via Tailwind CSS — tested on Chrome/Safari desktop and mobile
Offline tolerance	Mobile floor views queue stage updates to localStorage; flushed to Supabase when navigator.onLine fires true
Security	Supabase RLS on all tables; no sensitive logic in client-side code; JWT validated server-side via Next.js middleware
Performance	Next.js RSC for web views (fast initial load); Supabase Realtime for floor updates (no polling)
Solo build scope	All features scoped to what one developer can deliver; no CI/CD pipeline or test suite required for MVP

6. Recommended Build Order (Solo Developer)

Given a solo developer, build in this sequence to avoid blocking yourself:

1. Supabase schema + RLS policies — all other work depends on this foundation
2. Auth flow + Next.js middleware — role-based routing before building any screen
3. Production Planner web flow — PO entry form + BOM Edge Function
4. Floor mobile view — job card queue + stage buttons + offline queue logic
5. QC form + inventory deduction on material release
6. Accounts web flow + PDF challan generation
7. Director analytics dashboard — read-only aggregates, build last

7. Out of Scope — Version 1.0



These items are explicitly deferred. Do not build them in the MVP phase.

- GST / E-Way Bill portal integration — generate a PDF that looks like an invoice instead
- QR code scanning — users select orders from a list, not by scanning physical QR codes
- Email or SMS notifications — in-app status updates are sufficient for MVP
- Multi-tenancy — single factory installation only
- Automated testing suite — manual QA acceptable for MVP

